# JOB PORTAL MANAGEMENT SYSTEM

*School of Computing*

**Bachelor of Technology**
**in**
**Computer Science & Engineering**

**By**

**K.Sai Ganesh (VTU25201)**

*23UECS0302*
*Full Stack Application Development*
*Slot : E2-B13*


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
**SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY**
**(Deemed to be University Estd u/s 3 of UGC Act, 1956)**
**CHENNAI 600 062, TAMILNADU, INDIA**

**May, 2026**

# BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”Job Portal Management System” by K.Sai Ganesh (VTU25201) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

**Signature of Faculty**

**Dr. S Manohar M.E.,PhD**

**Assistant Professor**

**Computer Science & Engineering**

**School of Computing**

**Vel Tech Rangarajan Dr.Sagunthala R&D**

**Institute of Science and Technology**

**May, 2026**

**Signature of the Course Coordinator**

**Dr. Sumithra M**

**Assistant Professor (Senior Grade)**

**Computer Science & Engineering**

**School of Computing**

**Vel Tech Rangarajan Dr.Sagunthala R&D**

**Institute of Science and Technology**

**May, 2026**

# DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed. The project successfully fulfills the criteria required for the completion of the Full Stack Application Development module.

(K.Sai Ganesh)

Date:06/05/2026

# ABSTRACT

The Job Portal Management System is a full-stack web application designed to modernize recruitment by bridging the gap between employers and job seekers through a centralized, automated, and secure platform. Employers can post job openings enhanced with interactive geographic map integration, enabling precise office location targeting.The system improves communication and user engagement with an integrated floating chatbot and a visually appealing glassmorphism interface.Security is a core pillar, implemented through OTP-based email verification.Persistent data management is handled with MySQL,the system minimizes recruitment inefficiencies and unauthorized access. Overall, this project delivers a high-performance, scalable, and user-friendly solution tailored to the evolving needs of educational institutions and corporate hiring.

**Keywords:** Job Portal, Web Application, Spring Boot, Java, OTP Verification, Interactive Maps, Chatbot, MySQL, FSD.

# LIST OF FIGURES

# LIST OF TABLES

# LIST OF ACRONYMS AND ABBREVIATIONS

| | |
|---|---|
| UI | User Interface |
| UX | User Experience |
| API | Application Programming Interface |
| HTTP | HyperText Transfer Protocol |
| OTP | One Time Password |
| DB | Database |
| CRUD | Create, Read, Update, Delete |
| JPA | Java Persistence API |
| MVC | Model View Controller |
| CSS | Cascading Style Sheets |
| HTML | HyperText Markup Language |

# TABLE OF CONTENTS

# Chapter 1

# INTRODUCTION

## 1.1 Introduction

The Job Portal Management System is a robust, dynamic web-based application designed to radically simplify and automate the recruitment and job application process. Traditional methods typically involve the manual tracking of physical or emailed resumes, fragmented databases, and disconnected communication channels. These outdated methodologies frequently lead to severe inefficiencies in handling large pools of job seekers and managing multiple job postings across varying corporate departments. In the modern era, organizations require a unified ecosystem that handles data persistence, geographical context, and candidate verification automatically. This system provides an end-to-end centralized platform where employers and recruiters can create detailed job postings equipped with precise interactive map locations. Candidates, in turn, can view, filter, and apply for these posi-

tions seamlessly from their personalized dashboards. The project goes beyond basic CRUD operations by improving user communication via an integrated support chatbot and ensuring strict user validity through real-time OTP email verification powered by SMTP protocols. The application is developed entirely using Java, the Spring Boot ecosystem [1], Thymeleaf [3], and MySQL [2], resulting in a highly scalable, secure, and modern solution for the recruitment industry.

## 1.2 Aim of the Project

The primary aim of this project is to conceptualize, design, and develop a highly secure, web-based Job Portal Management System that fundamentally simplifies the hiring process. The overarching goal is to provide a unified, centralized platform where employers can post comprehensive jobs featuring geographical map data, and where job seekers can seamlessly track their application statuses. By enforcing email-based verification and utilizing a modern tech stack, the project aims to eliminate spam, enhance user experience, and provide a professional digital environment.

## 1.3 Scope of the Project

The scope of the Job Portal Management System is extensive, intending to provide a complete digital ecosystem for recruitment. The system allows employers to post, update, delete, and manage job listings with Leaflet map integrations, while users can securely register via email OTPs, upload their resumes via Multipart file handling, and apply with a single click. It ensures the highly efficient handling of complex applicant relational data and enforces robust security configurations using Spring Security. The scope also includes the integration of modern UI/UX principles, such as dynamic CSS animations, glassmorphism design elements, and universally accessible floating chatbots, to dramatically improve user retention and satisfaction across the platform.

## 1.4 Framework & Technologies

The system is heavily anchored in the Spring Boot framework, which provides robust inversion of control and dependency injection. It leverages Thymeleaf [3] for server-side HTML rendering to ensure SEO compliance, Java 17 for the backend business logic, and MySQL [2] for secure, scalable data persistence. Front-end technologies include HTML5, CSS3, JavaScript [8], and external APIs like Leaflet.js [4]

and OpenStreetMap for interactive mapping. Figure 1.1 shows the sys-

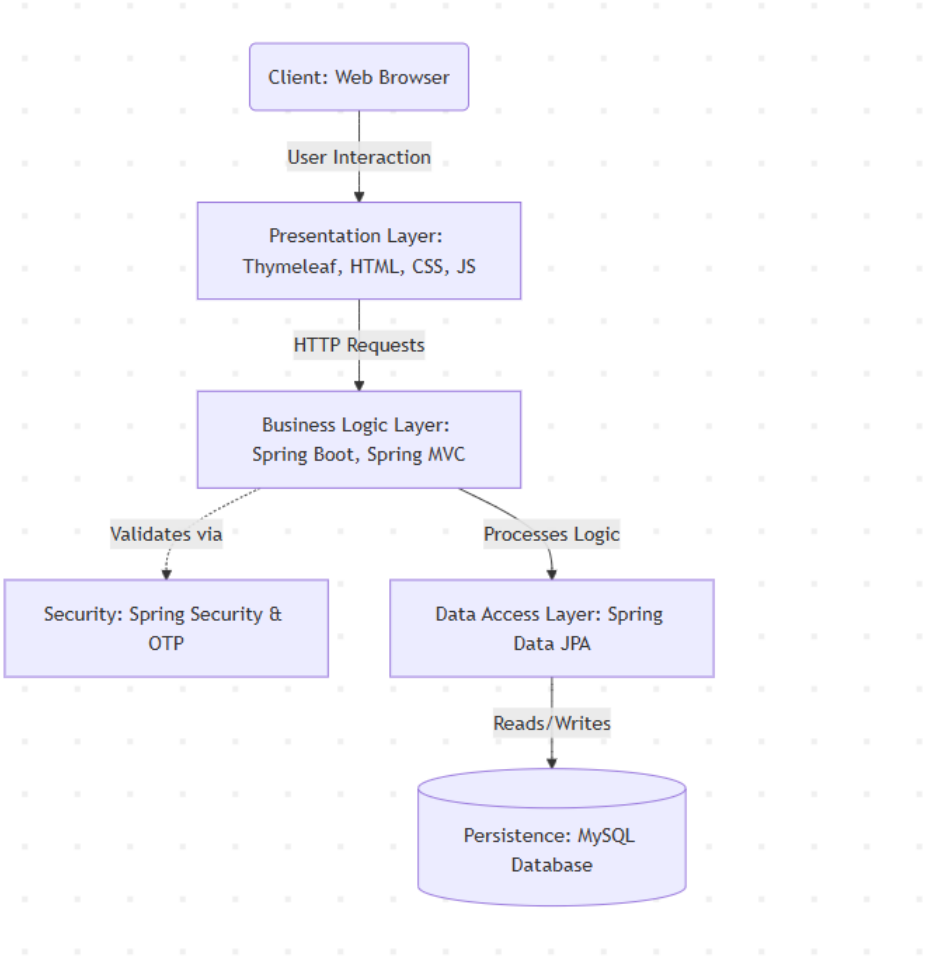



Figure 1.1: System Framework Architecture

tem architecture of the proposed application, which follows a layered design. Users interact with the system through a web browser, and their requests are handled by the presentation layer using HTML, CSS, JavaScript, and Thymeleaf. These requests are then processed in the business logic layer built with Spring Boot and Spring MVC.Security is managed using Spring Security and OTP verification to ensure safe access.This uses Spring Data JPA to communicate with the MySQL database,where all the data is stored.

# Chapter 2

# SURVEY OF SIMILAR APPLICATION IN MARKET

Several massive job portal platforms are currently available that help connect recruiters and job seekers efficiently. However, analyzing these existing systems reveals key areas where tailored, localized solutions can offer significantly better user experiences. LinkedIn [5] acts as the current global standard, allowing users to build extensive professional networks, share content, and apply for jobs globally. While incredibly feature-rich, its broad social-media focus can be highly distracting for a user who simply wants a streamlined application process. Indeed [6] acts as a massive global aggregator for job postings, scraping thousands of sites to provide a singular feed of opportunities. While it offers easy-apply features, the platform is notorious for high volumes of spam applications and unverified employer accounts, leading to a degraded trust ecosystem. Naukri.com [7] focuses heavily on

the Asian and specifically Indian market, allowing recruiters to filter through massive resume databases using complex queries. However, the user interface remains largely static and lacks modern design elements, often leading to a cluttered user experience. Although these platforms offer highly advanced features, they are often overwhelmingly complex for localized, institutional, or university-based recruitment. Furthermore, they frequently lack direct interactive integrations like embedded map pickers for precise office locations, relying instead on manual address entry. The proposed Job Portal Management System directly addresses these gaps by providing a highly customized, visually engaging, and highly secure alternative specifically tailored for streamlined employer-to-student recruitment with guaranteed user verification via OTPs.

# Chapter 3

# FRAMEWORK DESCRIPTION

## 3.1 Frontend Implementation

### 3.1.1 Environment Setup

The frontend of the project is developed using basic web technologies such as HTML, CSS, and JavaScript, tightly integrated with the Thymeleaf server-side template engine. A suitable development environment like Eclipse is used for coding and editing files. The application is executed on a web browser, and necessary tools such as a local server and the Spring Boot framework are configured to run the project smoothly [1].

### 3.1.2 Structure of the Project

The Job Portal Management System follows a structured Spring Boot architecture. The **src/main/java** directory contains the core application logic such as controllers and services.

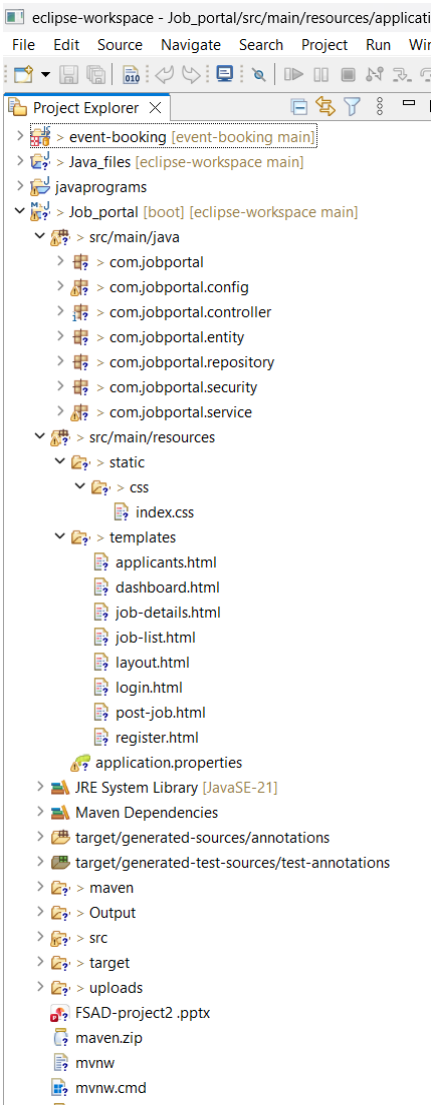



Figure 3.1: FrontEnd Structure

The **src/main/resources** folder includes static files and Thymeleaf templates for the user interface. The **application.properties** file is used for configuration, including database and SMTP server settings for OTPs. The **pom.xml** manages project dependencies. This structure ensures better organization, scalability, and easy maintenance.

### 3.1.3 Execution Procedure

To run the frontend, the project is first started using the backend

server (Spring Boot). Once the server is running, the application can be accessed through a web browser using the localhost URL (e.g., **http://localhost:8080**). Users can then interact with the system by navigating through different pages such as job listings, registration, OTP verification, and interactive map forms [2].

## 3.2 Backend Implementation

### 3.2.1 Environment Setup

The backend of the project is developed using Java with the Spring Boot framework. Required tools include JDK 17, an IDE such as Eclipse, and a database system like MySQL [3]. Dependencies are managed using Maven, and the project is configured with necessary libraries for web development, database connectivity, email dispatching, and server execution [1].

### 3.2.2 Structure of the Project

The backend of the Job Portal Management System follows a layered architecture to ensure scalability, maintainability, and efficient handling of application logic. The **controller** package contains classes such as **AuthController**, **StudentController**, and **EmployerController**, which handle HTTP requests and manage communication between the

client and server.

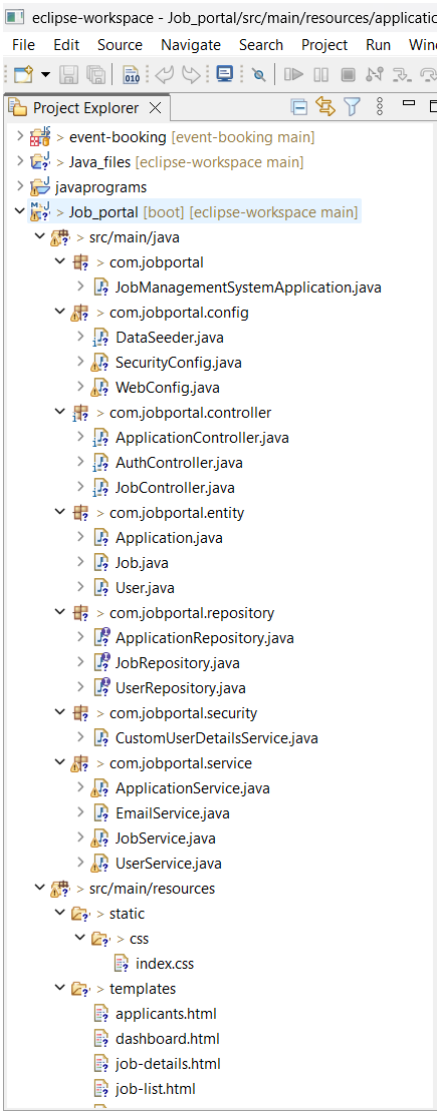



Figure 3.2: BackEnd Structure

The project follows a layered architecture using Spring Boot. The **controller** package handles HTTP requests, the **service** package contains business logic (like OTP validation), and the **repository** package manages database operations. The **entity** package defines entity classes such as **User**, **JobPosting**, and **JobApplication**.

The **config** package is used for application configuration (such as

Spring Security), while the **exception** package handles errors. The **resources** folder includes templates (HTML pages), static files, and **application.properties** for configuration. The **pom.xml** file manages dependencies, and the **target** folder stores compiled files.

### 3.2.3 Execution Procedure

To execute the backend, the project is run using Spring Boot through the IDE or by using Maven commands. Once the application starts, the embedded Tomcat server runs on a default port (e.g., 8080). The backend then handles requests from the frontend, processes data, generates emails, and interacts with the database to perform required operations.

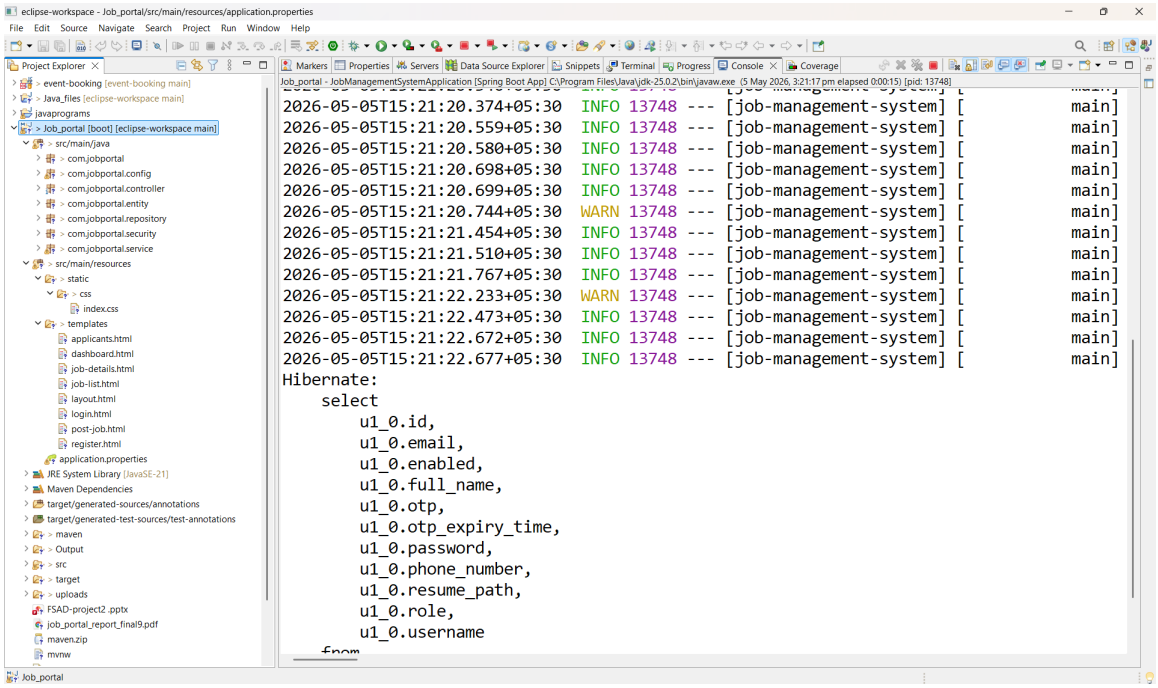



Figure 3.3: Execution

Figure 3.3 represents the execution of the backend system using the

Spring Boot framework. It displays the application startup logs, indicating successful initialization of server components, database connections, and configuration settings required for handling client requests.

## 3.3 Database Implementation

### 3.3.1 Database Configuration

The database used for this project is MySQL. It is configured by creating a database and connecting it with the Spring Boot application using properties such as URL, username, and password in the application configuration file. Spring Data JPA and JDBC are used for connectivity, and the system ensures proper data storage and retrieval [3], [4].

### 3.3.2 Schema Structure

The database schema is designed to store and manage data related to users, job postings, and applications. It includes tables with relationships to maintain data consistency and avoid redundancy. Primary keys are used to uniquely identify records, and foreign keys are used to establish relationships between tables.

### 3.3.3 Tables created

The main tables created in the database include the **users** table, **jobs** table, and **job_applications** table. The **users** table stores user details, the **jobs** table stores information about different job postings, and the **job_applications** table records submission statuses. These tables work together to support the overall functionality of the system.

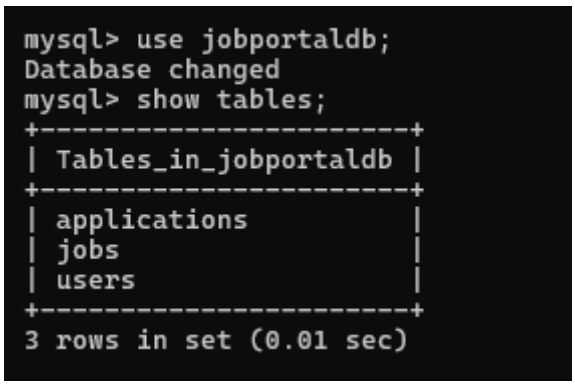



Figure 3.4: Tables in database

The tables in the job portal database are **users** (for storing user credentials), **jobs** (for storing job details), and **job_applications** .

**users table:**

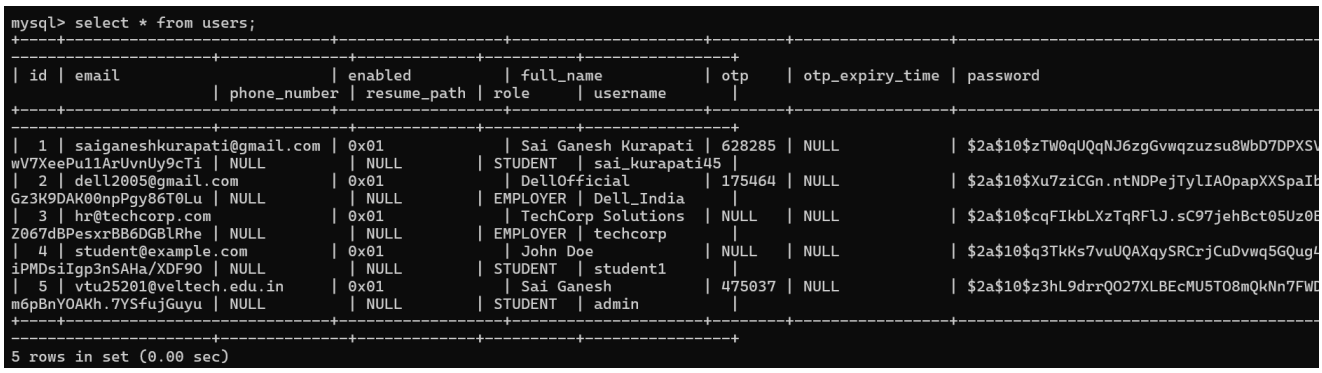



Figure 3.5: Users Table in database

Maintains user information including name, email, encrypted password, role (employer/student), company name, resume path, and ac-

count verification status. This data is used for secure login verification.

**jobs table:**

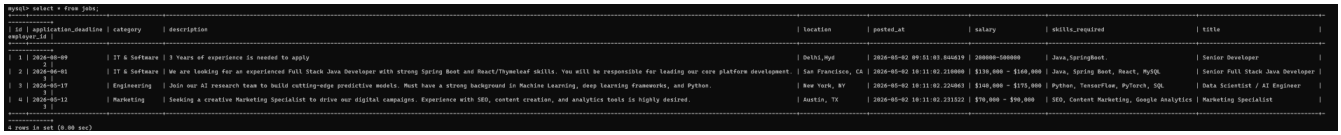
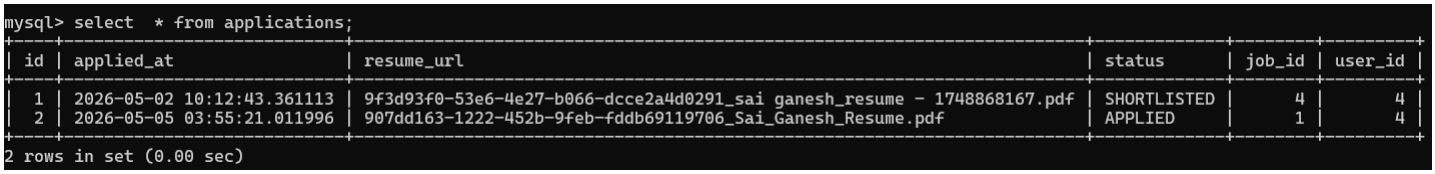


Figure 3.6: Jobs Table in database

Contains details of all job postings such as title, description, category, reverse-geocoded map location, salary range, and minimum experience required.

**job_applications table:**



```
mysql> select  * from applications;
+----+----------------------------+----------------------------------------------------------------------------------+-------------+--------+---------+
| id | applied_at                 | resume_url                                                                       | status      | job_id | user_id |
+----+----------------------------+----------------------------------------------------------------------------------+-------------+--------+---------+
|  1 | 2026-05-02 10:12:43.361113 | 9f3d93f0-53e6-4e27-b066-dcce2a4d0291_sai ganesh_resume - 1748868167.pdf | SHORTLISTED |      4 |       4 |
|  2 | 2026-05-05 03:55:21.011996 | 907dd163-1222-452b-9feb-fddb69119706_Sai_Ganesh_Resume.pdf                       | APPLIED     |      1 |       4 |
+----+----------------------------+----------------------------------------------------------------------------------+-------------+--------+---------+
2 rows in set (0.00 sec)
```

Figure 3.7: Job Applications Table in database

Stores information about user applications, including the exact time of application, dynamic application status (Pending, Shortlisted, Rejected), and links to the corresponding user and job posting.

# Chapter 4

# METHODOLOGIES

## 4.1 Project Architecture

The system architecture rigorously follows the Spring MVC (Model-View-Controller) paradigm. The View layer, comprised of Thymeleaf templates and HTML, captures user inputs and displays dynamic data. The Controller layer (e.g., AuthController, JobController) intercepts these incoming HTTP requests, performs initial validation, and directs them to the Service layer. The Service layer applies critical business rules, such as verifying OTP timeouts, ensuring salary constraints, and handling file I/O operations for resumes. Finally, the Repository layer interacts with the MySQL database to persist or retrieve the requested data.

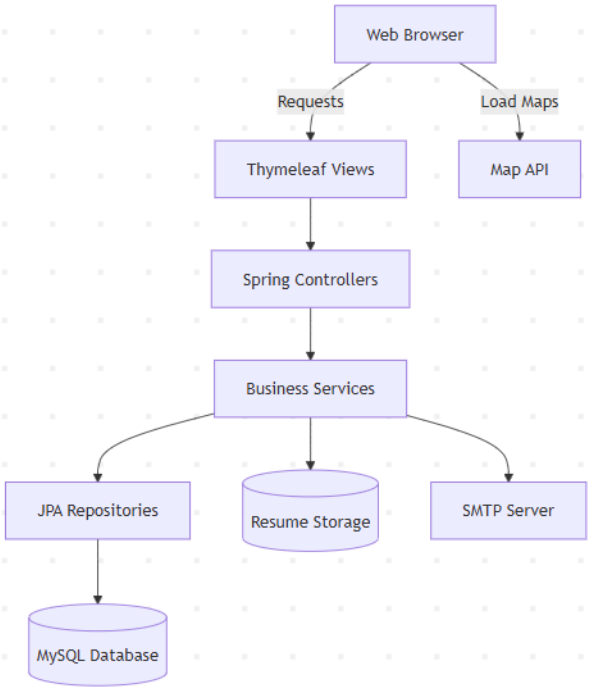



Figure 4.1: High-Level MVC Project Architecture

Figure 4.1 shows the system workflow of the Job Portal application. Users interact with the system through a web browser, and their requests are handled by Thymeleaf views and Spring controllers. The business services layer processes the core logic and connects to different components such as JPA repositories for database operations, resume storage for file handling, and an SMTP server for sending emails. The system also integrates a map API to display location-related information. All data is stored and retrieved from the MySQL database, ensuring smooth and efficient system functionality.

## 4.2 DataFlow Diagram

The data flow initiates the moment a user attempts to register. Upon submitting the form, the system generates a 6-digit OTP, suspends the database save, and triggers an SMTP email via Google servers. Once the user inputs the correct OTP, the data flows to the repository and the user is persisted. Post-login, Employers can input job data via interactive maps, which triggers JavaScript fetch API calls to OpenStreetMap, saving exact geographic strings to the DB. Students then fetch this data through GET requests, viewing the embedded map, and submit application payloads which flow back to the Employer's personalized dashboard.

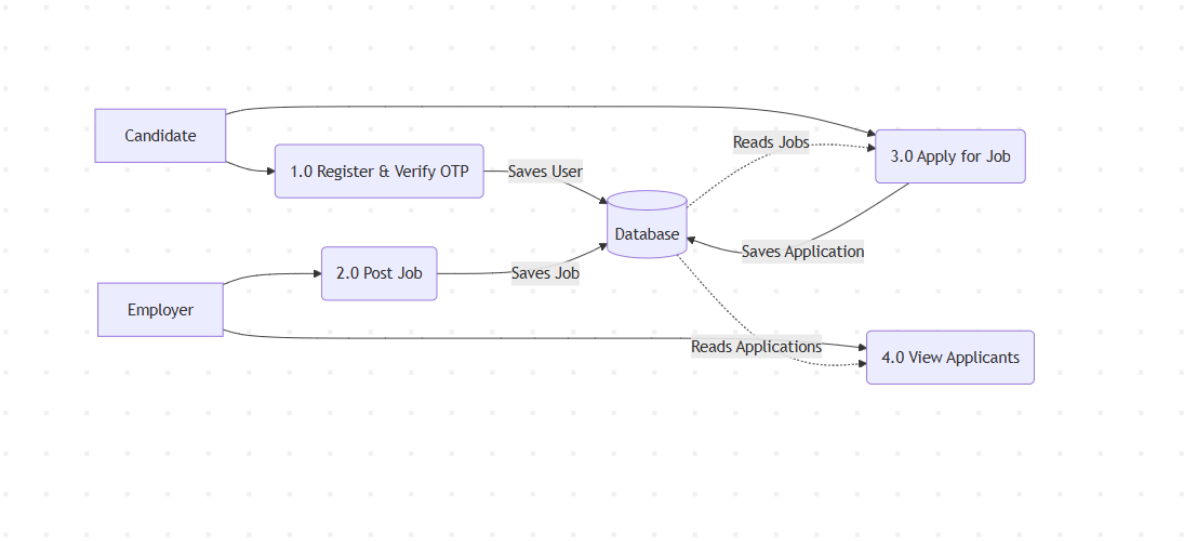



Figure 4.2: System Data Flow Diagram (DFD)

Figure 4.2 shows the data flow of the Job Portal system. The candidate first registers and verifies their account using OTP, and the details

are stored in the database. The employer can post jobs, which are also saved in the database. Candidates can then view available jobs and apply for them, with the application data being stored. Employers can later view the applications submitted by candidates. This process shows how data flows between users and the database in the system.

## 4.3 Module 1: Authentication & Security Module

This module acts as the impenetrable gatekeeper of the system. It handles the multi-step registration process and ensures absolute user validity. To prevent bot/spam accounts, it enforces an OTP verification system sent directly to the user's provided email. It heavily utilizes Spring Security to intercept unauthorized access, encrypt passwords using BCrypt hashing algorithms, and manage active HTTP login sessions seamlessly.This module ensures that only authenticated and authorized users can access system functionalities.It also enhances overall system reliability by preventing unauthorized access and data breaches.

## 4.4 Module 2: Employer Module

This module empowers recruiters and corporate entities with the ability to post and manage jobs.A standout feature of this module is

the fully interactive Leaflet Map integration. Recruiters can visually interact with a map, clicking to drop a pin on their exact office location. The system then automatically translates those map coordinates into readable city and state names via reverse geocoding REST APIs, storing this precise data for job seekers.This feature improves location accuracy and helps job seekers easily identify job locations.It also enhances the overall user experience by providing a more interactive and intuitive job posting system.

## 4.5 Module 3: Candidate Application Module

This module provides the core interface for job seekers and students. It allows them to securely browse dynamic job listings, utilize search filters, view geographic map markers of the job location, and securely upload PDF resumes.Chatbot widget designed to assist candidates with basic navigation, providing simulated, instantaneous support across all pages.This module enhances user engagement by providing easy access to relevant job opportunities.It also improves usability by offering real-time assistance and a seamless application experience.

# Chapter 5

# RESULTS AND DISCUSSIONS

The Job Portal Management System was successfully developed and tested. The system enables job seekers to view available job postings, apply online, and securely track their application statuses, ensuring a highly convenient and paperless recruitment process. Employers can efficiently create, update, and manage job postings utilizing interactive maps, as well as monitor candidate applications through a centralized dashboard interface.

The implementation demonstrates that the system effectively reduces manual HR work, improves hiring accuracy, and provides a centralized platform for managing corporate and campus recruitment. The user interface is simple, responsive, and easy to navigate, offering a smooth experience for both job seekers and employers. The backend processing ensures proper validation of data—including strict OTP email verification—and secure storage in the MySQL database, maintaining data integrity and reliability. Furthermore, the system provides

a strong foundation for future enhancements and scalability. Additional features such as real-time WebSocket notifications, AI-powered resume parsing, automated skill matching, and advanced analytics can be incorporated to further improve functionality. This flexibility ensures that the system can evolve based on institutional needs, making it a sustainable and long-term solution for efficient recruitment management.

**System Comparison table:**

Table 5.1: System Comparison

| **Feature** | **System Support** |
|---|---|
| Online Job Applications | Yes |
| OTP Email Verification | Yes |
| Interactive Map Integrations | Yes |
| Role-Based Dashboards | Yes |
| Manual Work Reduction | High |

**Table 5.1 shows** the key features of the Job Portal Management System, highlighting its support for digital applications, strict OTP security, and map integrations. It ensures centralized data handling and reduces manual recruitment work, improving overall efficiency and hiring accuracy.

# Login Page:

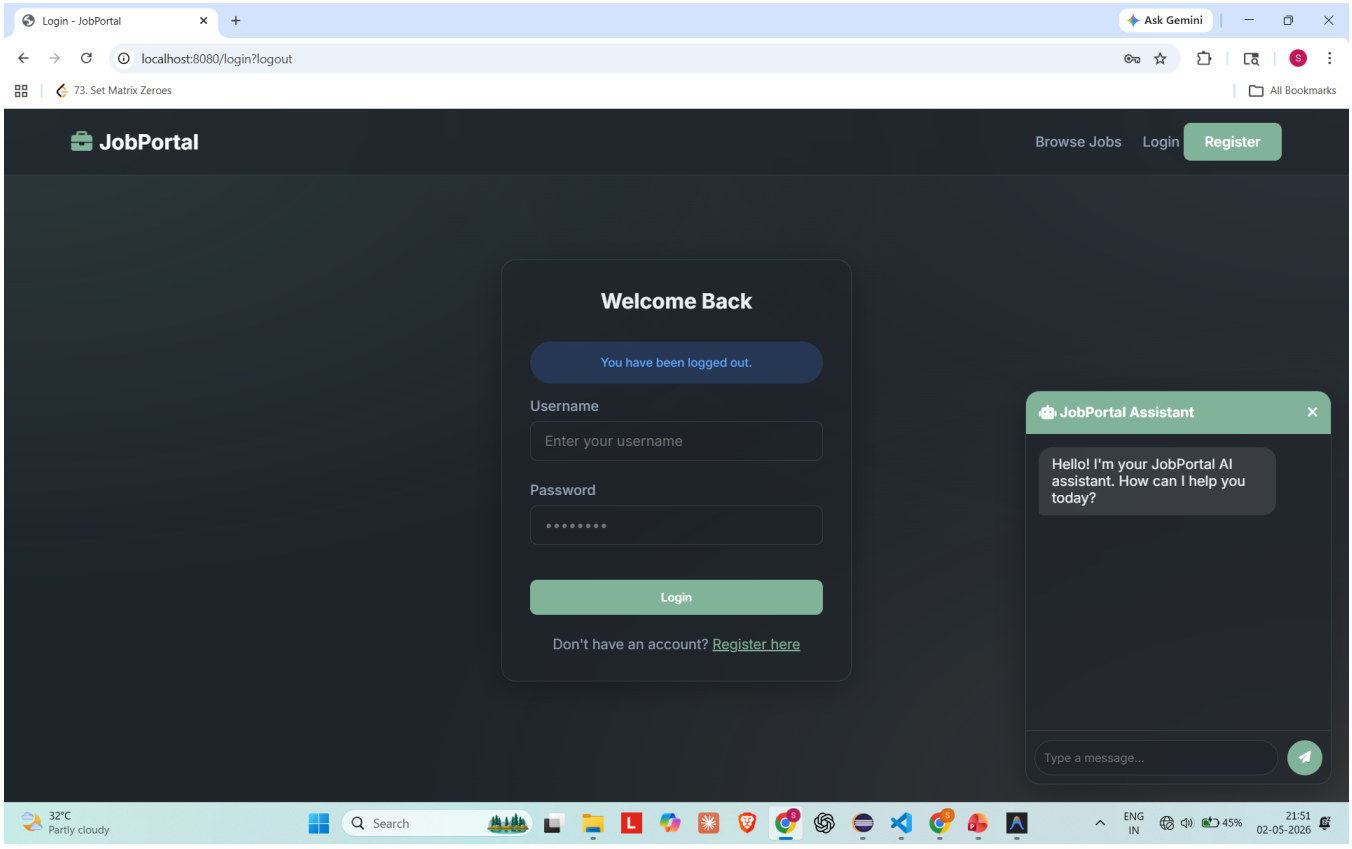



Figure 5.1: Login Page

**Figure 5.1** displays the login interface of the system where users enter their credentials to access their accounts. It ensures secure authentication via BCrypt password hashing and provides access to personalized job portal features after successful login.

## Job Display Page:

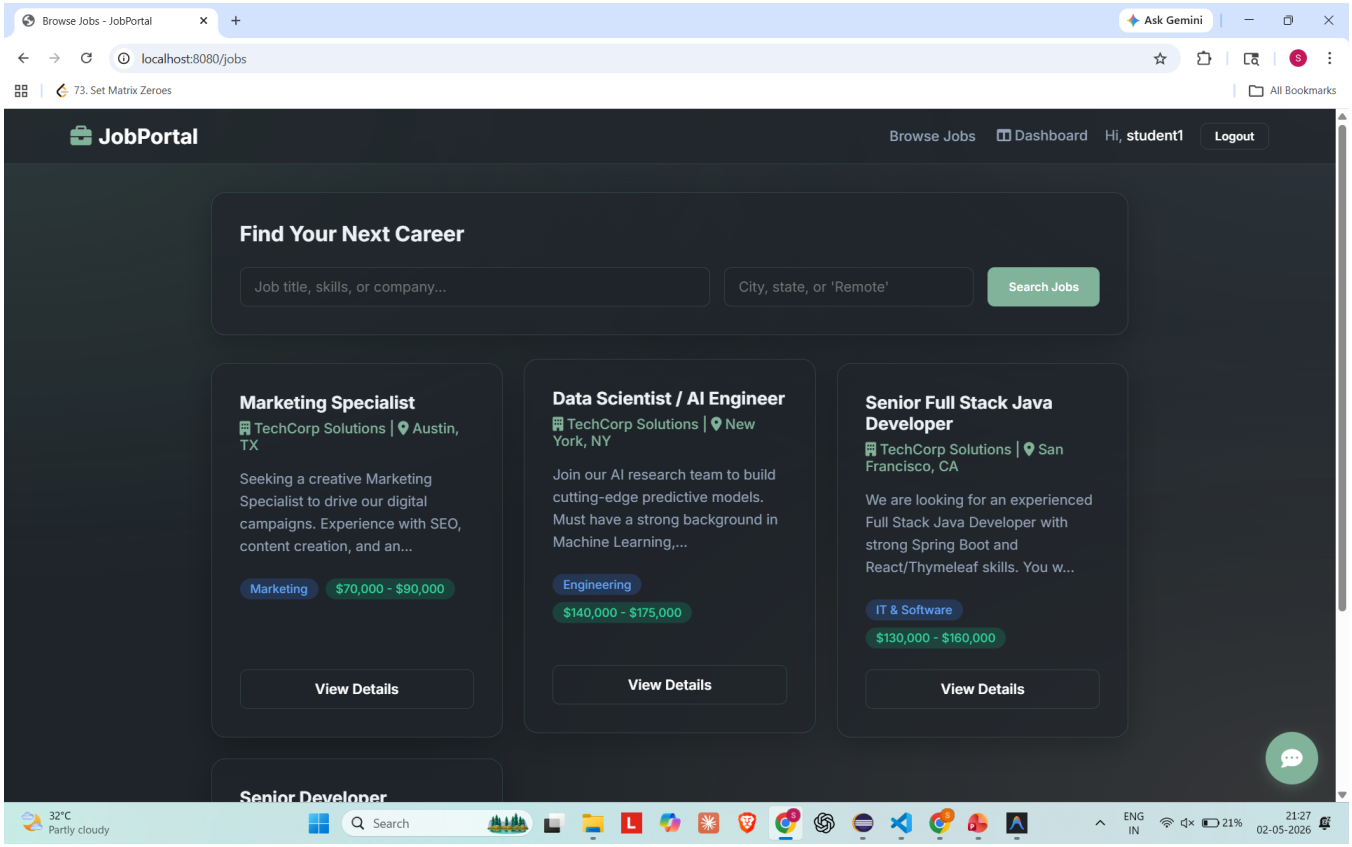



Figure 5.2: Jobs Display Page

**Figure 5.2** illustrates the browse jobs page where users can explore available positions along with details such as salary, required experience, and geographic location. It provides an easy interface with dynamic filters for users to select jobs for application.

## Registration Page:

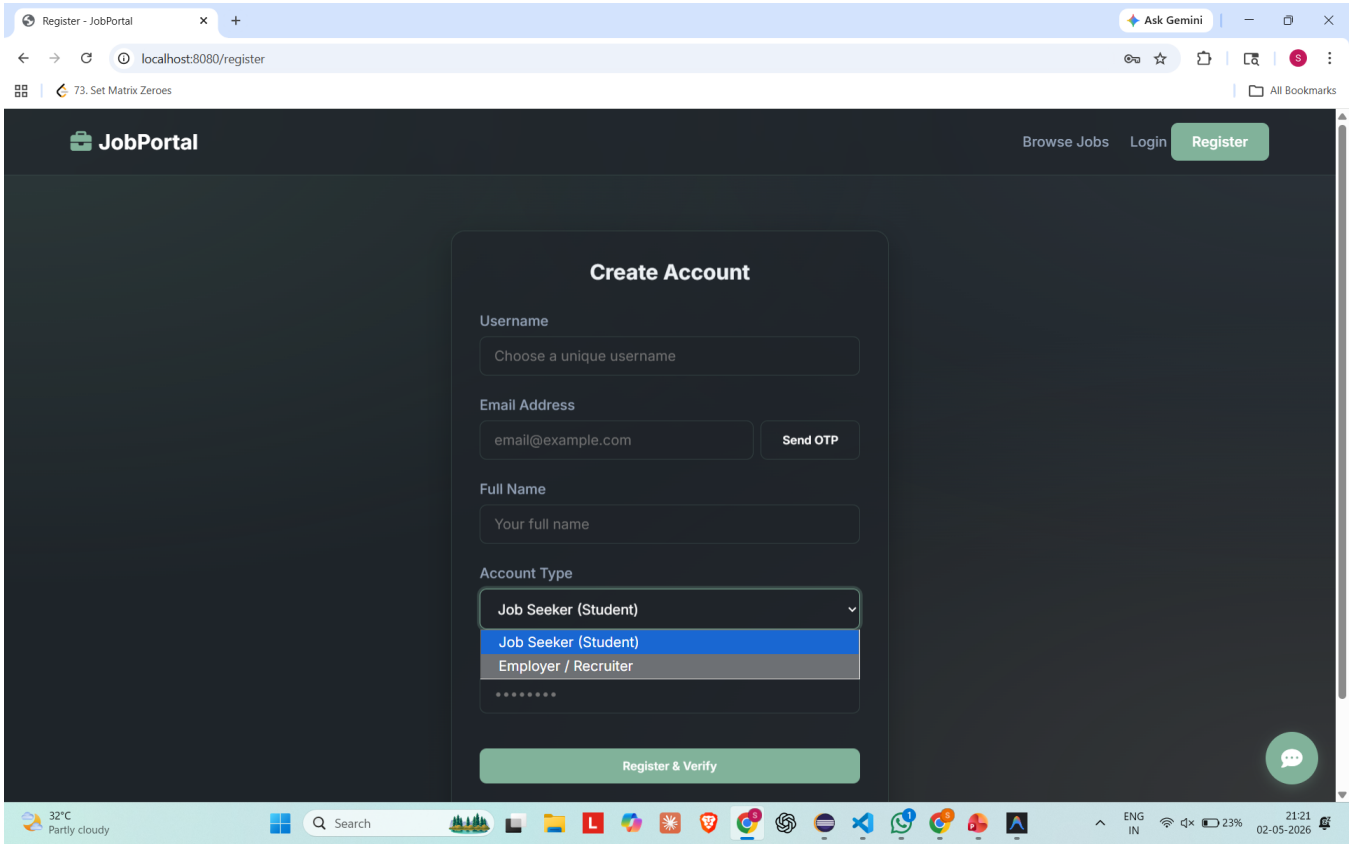



Figure 5.3: Register Page

**Figure 5.3** shows the registration page that allows new users to create an account.This triggers the OTP email verification process to ensure absolute system security.

## Interactive Map Job Posting:

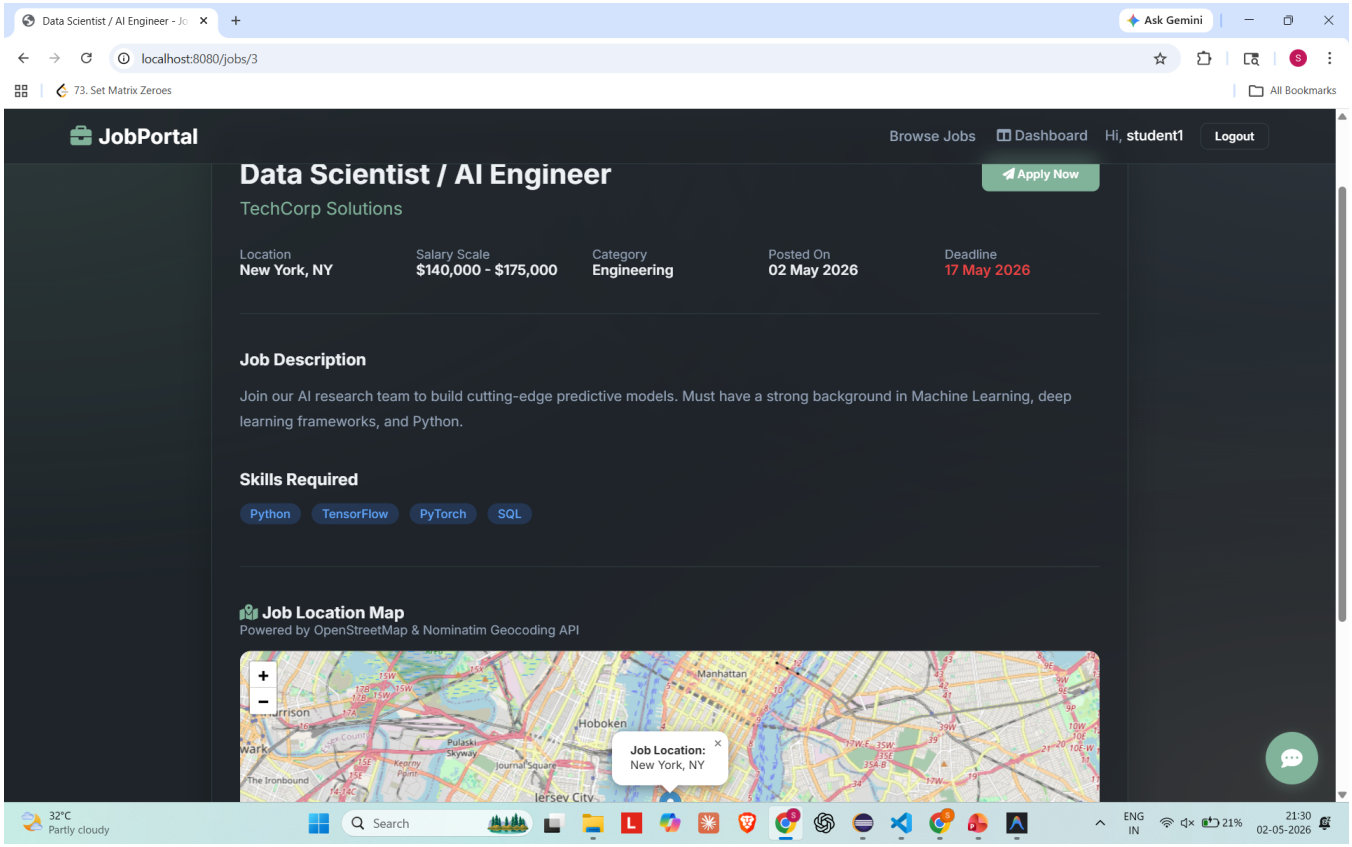



Figure 5.4: Interactive Map Job Posting

**Figure 5.4** shows the interactive Leaflet map interface utilized during job creation. Employers can directly drop a pin on their exact office location, which the system automatically translates into readable city and state coordinates via reverse-geocoding.

## Role-Based Dashboards:

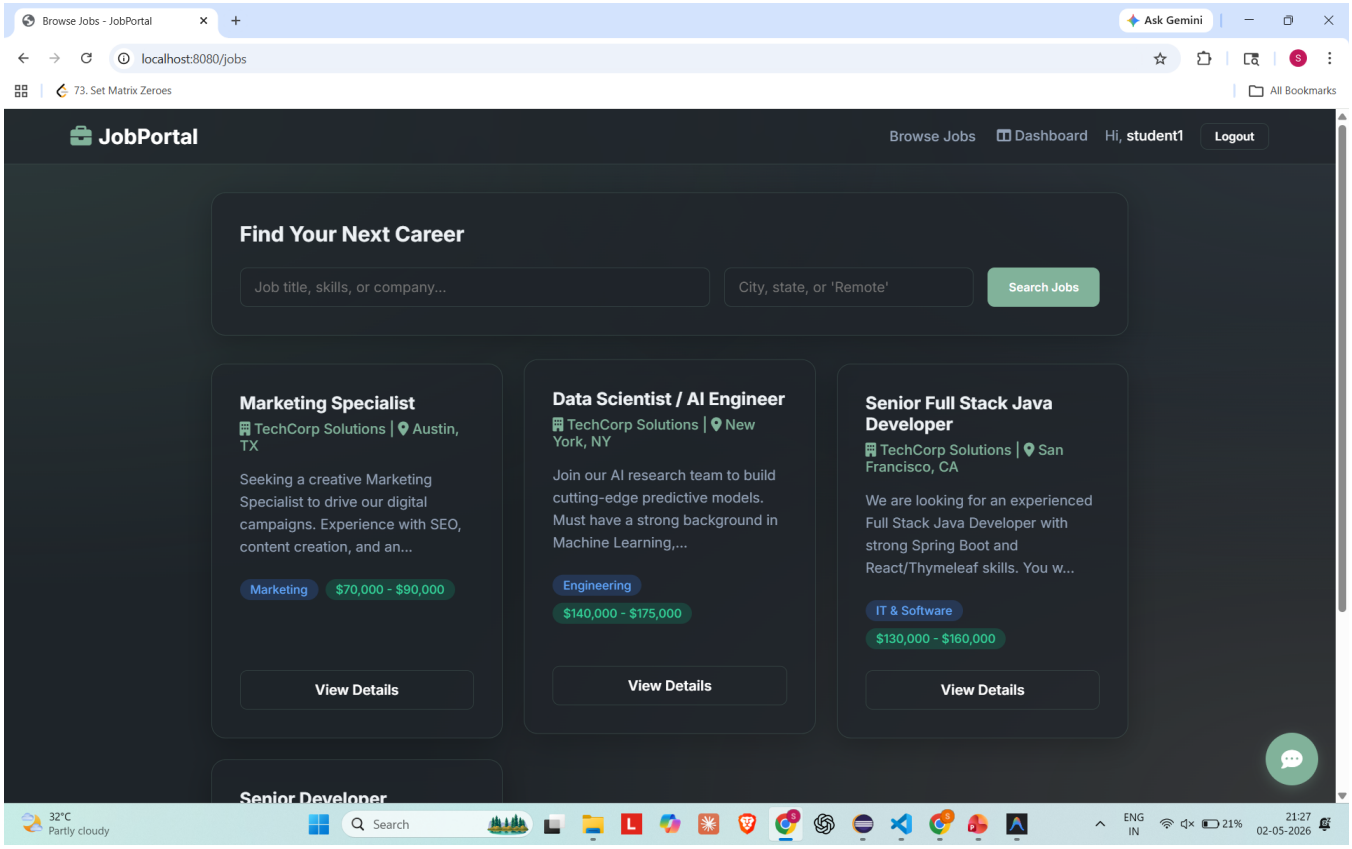



Figure 5.5: Role-Based Dashboard

**Figure 5.5** shows the personalized dashboard interface. For employers, it enables them to manage created jobs and monitor candidate submissions. For job seekers, it allows them to upload resumes, view shortlisting statuses, and withdraw active applications.

# Chapter 6

# CONCLUSION AND FUTURE ENHANCEMENTS

## 6.1 Conclusion

The Job Portal Management System undeniably proves to be an exceptionally secure, robust, and modern platform for connecting recruiters and candidates. By integrating advanced features like SMTP OTP validation, interactive Leaflet mapping, and a highly animated, glassmorphism-inspired UI, it resolves traditional recruitment bottlenecks. The system's ability to compartmentalize user roles into distinct, secure dashboards highlights the power of Spring Security. Ultimately, the project stands as a highly capable Spring Boot application demonstrating complete full-stack engineering proficiency, from database schema design to frontend DOM manipulation.

## 6.2 Future Enhancements

Although the current system is highly capable, several future enhancements could elevate it to an enterprise-grade platform. The static Chatbot can be upgraded to utilize a Large Language Model (LLM) API (such as OpenAI) to provide dynamic, context-aware candidate support. Furthermore, implementing cloud-based resume parsing (via AWS Textract) would allow the system to automatically extract candidate skills from PDF uploads. Finally, integrating real-time WebSocket notifications would instantly alert users when an employer reviews their application, providing a truly real-time digital experience. Additionally, the system can be enhanced by implementing advanced analytics to provide insights into hiring trends and user behavior.A recommendation engine can be introduced to suggest suitable jobs to candidates based on their skills and preferences.Integration with mobile applications can further improve accessibility and user engagement.Enhanced security measures such as multi-factor authentication can strengthen data protection. Finally, scalable cloud deployment can ensure better performance and reliability for a large number of users.

# Chapter 7

# ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

## 7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The development of the Job Portal Management System required handling multiple architectural and technical challenges. The system integrates frontend, backend, and database technologies while ensuring secure file handling and role-based access. This makes it a clear example of solving Complex Engineering Problems (CEP).One of the major challenges addressed is ensuring secure authentication and data protection, which is achieved through OTP-based verification and controlled access mechanisms. Another complexity lies in designing a scalable architecture that can handle multiple concurrent users without performance degradation. The implementation of MVC architecture helps in separating concerns and improving system maintainability.

Additionally, integrating external services such as email servers and map APIs introduces challenges related to latency, error handling, and data consistency.

| Level | Attribute | Characteristics | WKs | Justification |
|---|---|---|---|---|
| WP1 | Depth of Knowledge | Requires in-depth engineering knowledge | WK3, WK4, WK5, WK6 | Requires strong knowledge of Spring Boot, Spring MVC, Spring Security, JPA, and database design for building a multi-role job portal system. |
| WP2 | Conflicting Requirements | Technical and non-technical conflicts | WK4, WK5, WK6 | Balances role-based access control with usability, ensuring employers and job seekers have separate secure and user-friendly workflows. |
| WP3 | Depth of Analysis | Analytical and design thinking | WK3, WK4, WK5 | The system involves entity-relationship modeling, REST API design, and integration of file upload and email notification services. |
| WP4 | Familiarity | Partially new problems | WK4, WK8 | Job matching and application tracking introduce modern challenges beyond traditional CRUD systems. |
| WP5 | Applicable Codes | Limited standard solutions | WK6 | Requires custom design for resume storage, application tracking, and dynamic job filtering. |
| WP6 | Stakeholder Needs | Multiple stakeholders involved | WK5, WK6, WK7 | The system handles requirements of job seekers, employers, and admins with secure role-based access. |
| WP7 | Interdependence | System-level integration | WK3, WK5, WK6 | Integrates frontend (Thymeleaf), backend (Spring Boot), database (JPA/MySQL), security, and email services into one system. |

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Table 7.1 shows how the project addresses different aspects of Complex Engineering Problems, including knowledge depth, system integration, and stakeholder requirements.This demonstrates the system's ability to handle complex real-world engineering challenges effectively.

## 7.2 Mapping of the Project to Complex Engineering Activities (CEA)

The project includes multiple engineering activities such as OTP-based authentication, email notifications, and interactive UI features, which align with Complex Engineering Activities (CEA).These activities ensure the system is secure, efficient, and capable of handling real-world user requirements.

| EA No. | Attribute | Characteristics | Justification based on the Project |
|---|---|---|---|
| EA1 | Range of Resources | Use of diverse technologies | The project uses Spring Boot, Spring MVC, Spring Data JPA, MySQL, Thymeleaf, Spring Security, and JavaMailSender for full-stack development. |
| EA2 | Level of Interactions | Complex interactions between modules | The system manages interactions between job listings, applications, authentication, resume uploads, and email services. |
| EA3 | Innovation | Application of modern technologies | Includes role-based dashboards, dynamic filtering, file uploads, and email notifications. |
| EA4 | Consequences to Society and Environment | Impact on users and society | Improves employment access and provides a secure digital hiring platform. |
| EA5 | Familiarity | Beyond standard practices | Combines authentication, resume management, job tracking, and analytics into a single system. |

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

Table 7.2 shows the mapping of the project to Complex Engineering Activities, highlighting technologies used, system interactions, and real-world impact.This system improves efficiency, security, and user accessibility in the job recruitment process.

## 7.3 SDG Mapping (CEP Section)

This project aligns with multiple Sustainable Development Goals (SDGs), especially in promoting employment, innovation, and equal access to opportunities.

| SDG No. | SDG Title | Relevance to the Project |
|---|---|---|
| SDG 8 | Decent Work and Economic Growth | Connects job seekers with employers, improving employment opportunities. |
| SDG 9 | Industry, Innovation and Infrastructure | Supports digital recruitment using modern web technologies and automation. |
| SDG 10 | Reduced Inequalities | Provides equal access to job opportunities regardless of location. |
| SDG 16 | Peace, Justice and Strong Institutions | Ensures secure authentication and fair hiring using role-based access control. |

Table 7.3: SDG Mapping for the Project

Table 7.3 shows how the Job Portal system supports SDGs by improving employment access, promoting innovation, and ensuring secure digital processes.This highlights the system's contribution to sustainable development and inclusive digital transformation.

# References

[1] Spring Boot Documentation (2024). Spring Boot Official Guide. Available at: https://spring.io/projects/spring-boot

[2] Oracle Corporation (2024). MySQL Database Documentation. Available at: https://www.mysql.com

[3] Thymeleaf Documentation (2024). Server-side Java template engine. Available at: https://www.thymeleaf.org

[4] Leaflet (2024). An open-source JavaScript library for mobile-friendly interactive maps. Available at: https://leafletjs.com/

[5] LinkedIn Jobs (2024). Professional Job Portal Platform. Available at: https://www.linkedin.com/jobs

[6] Indeed (2024). Job Search Engine. Available at: https://www.indeed.com

[7] Naukri.com (2024). Indian Job Portal. Available at: https://www.naukri.com

[8] W3Schools (2024). HTML, CSS and JavaScript Tutorials.